MINI ASSESSMENT

API ERROR HANDLING AND SAFE JSON ACCESS

Name	Rohith M
Reg. No.	99230040751
Department	CSE – AIML

1 If an API returns 429, what should your program do?

FLOW OF HANDLING

API Response 429	Check Retry-After	Wait Backoff	Retry Safely

Answer	A 429 status indicates that too many requests have been sent to the API. The program should pause before making another request. If a **Retry-After** value is available, the program should follow it; otherwise, it can use an **exponential backoff** delay. The program should limit the number of retries and handle the situation safely if the request still fails.

2 If response.json() fails, what should your program do?

FLOW OF HANDLING

Response Received	Parse JSON Attempt	Failure Catch Error	Handle Safely

Answer	If **response.json()** cannot parse the response, the program should catch the parsing error instead of allowing the application to crash. It should check the response status and, if needed, look at the response text for debugging. The failure should be reported clearly, and the program should either continue safely or stop the current operation in a controlled manner.

MINI ASSESSMENT

API ERROR HANDLING AND SAFE JSON ACCESS

3 If a JSON key is missing, why might .get() be safer than []?

FLOW OF HANDLING

JSON Key may be absent	.get("key") None / default	Safe No KeyError	["key"] Error if absent

Answer	Using **data.get("key")** is safer when the JSON field may not be present. It returns **None**, or a default value if one is provided, instead of producing an error. On the other hand, **data["key"]** expects the key to exist and raises a **KeyError** when it is missing. Therefore, **.get()** is useful for optional or uncertain JSON fields.

QUICK COMPARISON

	Key exists	Key missing
data.get("key")	Returns value	Returns None / default
data["key"]	Returns value	Raises KeyError

KEY TAKEAWAY

Good API programs should be prepared for rate limits, invalid responses, and missing JSON fields so that unexpected errors are handled safely.